Matemática para Aprendizaje Automático

Notas de Clase — Precisión Numérica

Universidad Católica del Uruguay • 2026

> ***Referencias principales:*** *Trefethen y Bau 1997, Numerical Linear Algebra, lecciones 12 (condicionamiento), 13 (punto flotante), 14-15 (estabilidad). Higham 2002, Accuracy and Stability of Numerical Algorithms, caps. 1 y 2.* ***Referencias complementarias:*** *Golub y Van Loan 2013, Matrix Computations, sección 2.7 (punto flotante), sección 2.6 (condicionamiento de problemas matriciales); Wikipedia: IEEE 754*

Disclaimer: estas notas son un borrador en desarrollo, y pueden contener errores o imprecisiones. Se agradecen correcciones, sugerencias y comentarios para mejorar su calidad a `pablo.iturralde@ucu.edu.uy`

1. Análisis numérico

En las primeras semanas del curso trabajamos vectores, matrices, sistemas lineales, espacios propios y proyecciones de forma analíṗica. Esa perspectiva matemática es necesaria, pero no alcanza para entender qué ocurre cuando esos mismos problemas se resuelven de forma numérica (p.ej. en una computadora).

Esta área de estudio se conoce como análisis numérico, y se ocupa principalmente de tres preguntas:

1. **¿Cómo resolver un problema dado?** Por ejemplo, para resolver un sistema lineal $Ax = b$, podríamos calcular A^{-1} (con algún método, como la regla de Cramer) y luego multiplicar por b, o podríamos usar eliminación Gaussiana. Esto es un problema de diseño de algoritmos.
2. **¿Cuánto "cuesta" el algoritmo?** Para cada algoritmo, podemos analizar su complejidad computacional (gruesamente: la cantidad de cuentas que es necesario realizar para completar la tarea). Esto tiene implicancias sobre los recursos (tiempo, energía, hardware) necesarios para resolver el problema, y sobre la escalabilidad del método a problemas de gran tamaño. Esta es una pregunta central en el Aprendizaje Automático moderno, donde los conjuntos de datos y los modelos son cada vez más grandes, y el acceso a hardware y energía comienzan a ser factores limitantes.
3. **¿Qué tan precisos son los resultados?** Al resolver un problema de forma numérica, obtenemos una aproximación al resultado exacto. La precisión de esa aproximación depende de la representación de los números, de la sensibilidad del problema a perturbaciones en los datos, y de la estabilidad del algoritmo que usamos para resolverlo. Esta pregunta también tiene implicancias prácticas en el Aprendizaje Automático moderno, en general en relación a la pregunta anterior: si queremos usar hardware de bajo costo o bajo consumo energético, es posible que tengamos que trabajar con representaciones de menor precisión, lo que a su vez puede afectar los resultados de manera indeseable o inaceptable.

En estas notas se abordan aspectos relacionados con la última pregunta: la precisión de los resultados numéricos. Las mismas cubren tres ideas relacionadas:

1. **Representación:** cómo se almacenan números con precisión finita, y qué errores introduce eso.
2. **Condicionamiento:** qué tan sensible es un problema matemático a perturbaciones en los datos de entrada.
3. **Estabilidad:** si un algoritmo concreto amplifica los errores más de lo que el problema exige.

1.1 El problema de representar números reales

La recta real contiene infinitos números — de hecho, una cantidad no numerable de ellos. Sin embargo, cualquier sistema de almacenamiento digital dispone de una cantidad finita de memoria. Si asignamos b bits para representar un número, podemos distinguir a lo sumo 2^b valores diferentes. Esto implica que la inmensa mayoría de los números reales no pueden representarse de forma exacta: toda representación finita es necesariamente una aproximación.

Dado este presupuesto fijo de bits por número, surge una pregunta de diseño: ¿cómo conviene distribuir los valores representables a lo largo de la recta real? ¿Uniformemente? ¿Con mayor densidad en ciertas zonas?

Las secciones siguientes exploran dos respuestas a esta pregunta. La representación de punto fijo (sección 2.1) distribuye los valores de forma uniforme, lo que da una precisión absoluta constante pero un rango limitado. La notación científica o punto flotante (sección 2.2) los distribuye de forma logarítmica, sacrificando precisión absoluta uniforme a cambio de un rango mucho mayor y una precisión *relativa* aproximadamente constante. Esta segunda opción es la base de la representación que usan las computadoras modernas.

---

2. Aritmética con precisión finita: ejemplos en papel

Antes de hablar de computadoras, veamos qué pasa cuando hacemos cuentas con una cantidad limitada de dígitos decimales, algo que usualmente hacemos en papel.

2.1 Representación decimal de punto fijo

Supongamos un sistema que solo puede almacenar 5 dígitos decimales en formato de punto fijo: un dígito antes del punto y cuatro después. Es decir, todo número se representa como

$$\texttt{X.XXXX}$$

donde cada X es un dígito del 0 al 9. El rango representable va de 0.0000 a 9.9999, y podemos representar 10^5 números diferentes.

Errores de representación

Algunos números reales (casi todos) no tienen representación exacta en este sistema:

$$\pi = 3.14159265\ldots \quad \mapsto \quad 3.1416$$

$$\frac{1}{3} = 0.33333\ldots \quad \mapsto \quad 0.3333$$

Es decir, la representación es aproximada.

TODO: add svg with line showing real numbers and representable numbers, uniformly distributed.

Error absoluto y error relativo

Si x es el valor exacto y $\hat{x}$ su representación, definimos:

$$\text{error absoluto} = |x - \hat{x}|,$$

$$\text{error relativo} = \frac{|x - \hat{x}|}{|x|}, \qquad x \neq 0.$$

En un sistema con redondeo apropiado, el error absoluto de representación está acotado por la mitad de la distancia entre dos números representables (dos "vecinos"). Para un sistema como el propuesto, podemos esperar tener un error absoluto de hasta 0.00005 (la mitad de la última cifra) en cada número representado, independientemente de su magnitud. Es decir, el error absoluto está acotado.

Por otro lado, el error relativo varía con la magnitud del número. Para números grandes, el error relativo es pequeño, pero para números pequeños el error relativo puede ser grande.

En cómputo numérico suele importar más el error relativo.

Errores aritméticos

La representación aproximada introduce errores aritméticos. Por ejemplo, si intentamos calcular $3 \times \frac{1}{3}$, esperamos obtener 1.0000. Pero con nuestra representación:

$$3 \times 0.3333 = 0.9999,$$

y la resta $1.0000 - 3 * (1/3) = 0.0001$, que debería ser exactamente cero.

Vemos de los ejemplos anteriores que trabajar con precisión finita introduce errores de representación, que a su vez pueden causar errores aritméticos o resultados inesperados en las cuentas.

2.2 Notación científica con 5 dígitos/Punto flotante

Las representaciones de punto fijo como la dada anteriormente tienen dos limitaciones evidentes: su rango limitado y la precisión no uniforme.

En nuestro ejemplo anterior, el rango representable es estrecho (0.0000 a 9.9999 en nuestro ejemplo). Si necesitamos representar el número de Avogadro (6.022×10^{23}) o la masa del electrón (9.109×10^{-31}), estos quedan completamente por fuera del rango representable.

Adicionalmente, mantenemos una precisión absoluta fija sobre todo el rango, pero en la práctica usualmente queremos mantener una precisión relativa uniforme, independientemente de la magnitud del número, para poder trabajar de igual manera con números muy grandes y muy pequeños.

La notación científica resuelve esto redistribuyendo los dígitos disponibles. Con el mismo presupuesto de 5 dígitos por número, podemos asignar:

$$\texttt{X.XX} \times 10^{\pm\texttt{XX}}$$

Donde se usó:

- 3 dígitos a la **mantisa** (1 entero + 2 decimales): X.XX
- 2 dígitos al **exponente** (con signo): ±XX

Veamos algunos ejemplos de representación en este formato vs. punto fijo:

Valor	Punto fijo (X.XXXX)	Notación científica (X.XX × 10^±XX)
π	3.1416	3.14×10^0
1	1.0000	1.00×10^0
Número de Avogadro	no representable	6.02×10^{23}
Masa del electrón	no representable	9.11×10^{-31}
$\frac{1}{3}$	0.3333	3.33×10^{-1}

Rango vs. precisión

El rango ahora es enorme: desde 1.00×10^{-99} (menor valor positivo representable) hasta $9.99 \times 10^{+99}$ (mayor valor positivo representable), comparado con 0.0000 a 9.9999 en punto fijo.

El "precio" a pagar por este rango superior es de precisión: antes podíamos garantizar que el error absoluto era menor a 5×10^{-5}. Ahora esto no ocurre: solo podemos garantizar que el error absoluto es menor a $5 \times 10^{-3} \times 10^{\text{exponente}}$, lo que puede ser grande o pequeño dependiendo del número en cuestión y su exponente.

Error relativo acotado

La ventaja de la notación científica es que el error relativo queda acotado de forma uniforme. Si consideramos los números mayores a 1, la mantisa siempre tiene 3 cifras significativas, por lo que el error de representación es como máximo la mitad de la última cifra de la mantisa:

$$\text{error relativo} \lesssim \frac{0.005}{1.00} = 0.5\%$$

Esta precisión relativa uniforme es una propiedad deseable, pues permite garantizar que el error de representación no se amplifica desproporcionadamente para números pequeños o grandes.

Los mismos problemas persisten

Aunque la notación científica permite garantizar una precisión relativa aproximadamente uniforme y brinda un rango mucho mayor de representación, los problemas aritméticos de precisión finita vistos en la sección anterior siguen existiendo.

3. Punto flotante e IEEE 754

3.0 Representación binaria

Las computadoras representan información en binario. La idea de de la representación binaria es idéntica a la representación decimal, pero utilizando base-2, o dos valores posibles por dígito.

Consideremos el problema de representar un entero cualquiera, por ejemplo 112. En base decimal la representación surge de la descomposición:

$$\mathbf{112} = \mathbf{1} \cdot 10^2 + \mathbf{1} \cdot 10^1 + \mathbf{2} \cdot 10^0$$

Es decir, utilizamos las potencias de 10 como "lugar" para cada dígito. En base binaria, la representación surge de la descomposición:

$$\mathbf{1110000} = \mathbf{1} \cdot 2^6 + \mathbf{1} \cdot 2^5 + \mathbf{1} \cdot 2^4 + \mathbf{0} \cdot 2^3 + \mathbf{0} \cdot 2^2 + \mathbf{0} \cdot 2^1 + \mathbf{0} \cdot 2^0$$

La representación con cifras después del punto decimal implica simplemente admitir potencias negativas, donde el punto decimal separa los dígitos correspondientes a potencias positivas y negativas. Por ejemplo:

$$\mathbf{7.5} = \mathbf{7} \cdot 10^0 + \mathbf{5} \cdot 10^{-1}$$

En binario, 7.5 se representa como 111.1:

$$\mathbf{111.1} = \mathbf{1} \cdot 2^2 + \mathbf{1} \cdot 2^1 + \mathbf{1} \cdot 2^0 + \mathbf{1} \cdot 2^{-1}$$

3.1 De la notación científica al punto flotante binario

La idea del punto flotante es exactamente la misma que la notación científica, pero en base 2:

$$x = (-1)^s \cdot m \cdot 2^e$$

donde:

- s es el bit de **signo** (0 = positivo, 1 = negativo);
- m es la **mantisa** (o significando);
- e es el **exponente**.

3.2 El estándar IEEE 754

El estándar IEEE 754 (1985, revisado en 2008 y 2019) fija las convenciones para la representación de números en punto flotante, las operaciones aritméticas, y el manejo de casos excepcionales. Es la norma que siguen prácticamente todos los procesadores modernos.

La representación usa una cantidad fija de bits, divididos en tres campos:

- **1 bit** para el signo (0 = positivo, 1 = negativo)
- k **bits** para el exponente.
- p **bits** para la mantisa.

Por una cuestión de conveniencia, el exponente se representa como un número entero positivo, por lo que se introduce un sesgo (*bias*) igual a $2^{k-1} - 1$ para permitir exponentes negativos. Por ejemplo, si se trabaja con $k = 8$, se pueden representar los enteros en el rango $[0, 255]$, y el sesgo es 127, lo que permite representar exponentes en el rango $[-127, +128]$.

La mantisa es un número de punto fijo, con un único dígito entero y p bits fraccionarios. Para evitar redundancia en la representación, solo se admite que la mantisa esté en el rango $[1, 2)$. Por tanto, alcanza con 1 bit para representar su parte entera, y esta además siempre es 1, por lo que no es necesario almacenarla explícitamente (bit implícito) y solo se almacenan los bits fraccionarios de la mantisa.

A partir de lo anterior, la fórmula de conversión para números normalizados es:

$$\text{valor} = (-1)^s \times (1 + \text{mantisa}) \times 2^{(\text{exponente} - \text{bias})}$$

Ejemplo: Formato de 32 bits (single precision)

Si usamos 32 *bits* para representar un número, podríamos asignar 1 bit para el signo, 8 bits para el exponente, y 23 bits para la mantisa:[1]

Representación de punto flotante en IEEE 754. Crédito: By Fresheneesz at the English Wikipedia project, CC BY-SA 3.0

En este ejemplo, la distancia relativa entre números representables consecutivos puede verse en la siguiente figura. La misma refleja la naturaleza, aproximadamente uniforme, del espaciado en este tipo de representación.[2]

Ejemplo de conversión

Si trabajamos con 8 bits de mantisa (256 valores posibles) y 4 bits de exponente (16 valores posibles), el número representado por los bits $s = 0$, mantisa = 10100000 (en binario), y exponente = 1101 (es decir 13 en representación decimal), con bias = 7 es:

$$\text{valor} = (-1)^0 \times (1 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 0 \times 2^{-4} + 0 \times 2^{-5} + 0 \times 2^{-6} + 0 \times 2^{-7}) \times 2^{(13-7)} = (1 + 0.5 + 0.125) \times 2^6$$
$$= 2^6 + 2^5 + 2^3 = 64 + 32 + 8 = 104$$

3.2.1 Bits de mantisa → precisión relativa

Al igual que en la notación científica, la cantidad de dígitos (binarios) en la mantisa determina la precisión relativa de la representación. Para calcular la cantidad de dígitos decimales de precisión que se pueden garantizar a partir de p bits de mantisa, podemos usar la relación entre logaritmos en base 10 y base 2:

$$\text{dígitos significativos} \approx p \times \log_{10}(2) \approx p \times 0.301$$

Por ejemplo, 52 bits de mantisa dan $52 \times 0.301 \approx 15.7$ dígitos decimales de precisión. 10 bits dan ≈ 3 dígitos.

3.2.2 Epsilon de máquina

El **epsilon de máquina** se define formalmente como la diferencia entre 1 y el siguiente número representable mayor que 1. Es decir, es el menor número ε tal que $\text{fl}(1 + \varepsilon) \neq 1$, donde $\text{fl}(\cdot)$ denota la función de redondeo a la representación de punto flotante.

Este valor tiene la interpretación de ser el error absoluto máximo de representación para números cercanos a 1, y por la propiedad de error relativo uniforme de punto flotante, es una cota del error relativo máximo de representación para cualquier número representable (excepto cero).

Si utilizamos una representación con p bits de mantisa, el menor número representable mayor que 1 es $1 + 2^{-p}$, por lo que el epsilon de máquina es:

$$\varepsilon_{\text{mach}} = 2^{-p}$$

3.2.3 Bits de exponente → rango

Los k bits de exponente determinan el rango de valores representables. El valor máximo representable es aproximadamente:

$$\text{máx} \approx 2^{2^{k-1}}$$

Por ejemplo, con $k = 11$ bits de exponente, el valor máximo es del orden de $2^{1024} \approx 1.8 \times 10^{308}$.

3.2.4 Formatos comunes definidos en IEEE 754

El estándar define varios formatos. Los tres más utilizados:

Formato	Bits	Exp. (k)	Mantisa (p)	$\varepsilon_{\text{mach}}$	Rango máx.	Precisión decimal
Half (FP16)	16	5	10	9.77×10^{-4}	$\sim 6.5 \times 10^4$	~3
Single (FP32)	32	8	23	1.19×10^{-7}	$\sim 3.4 \times 10^{38}$	~7
Double (FP64)	64	11	52	2.22×10^{-16}	$\sim 1.8 \times 10^{308}$	~16

Más bits de mantisa implican más precisión; más bits de exponente implican más rango. Pero más bits en total implican más memoria necesaria para almacenarlos, más ancho de banda para transferir valores entre dispositivos, y mayor latencia en las operaciones (tiempos de cálculo).

3.2.5 Valores especiales y subnormales

El estándar IEEE 754 reserva ciertas combinaciones de bits para valores especiales:

- **Cero:** exponente = 0, mantisa = 0. El bit de signo distingue $+0$ y -0 (aritméticamente equivalentes, pero distinguibles en ciertas operaciones).
- **Infinito ($\pm\infty$):** exponente = todos 1s, mantisa = 0. Aparece en *overflow* (resultados demasiado grandes para representar) o división por cero.
- **NaN (*Not a Number*):** exponente = todos 1s, mantisa $\neq 0$. Resultado de operaciones indefinidas como $0/0$ o $\sqrt{-1}$ en los reales.

Los **números subnormales** (o denormalizados) son un conjunto de valores que cubren la zona entre cero y el menor número normalizado. Se representan con exponente = 0 y se asume que el bit implícito de la mantisa es 0 y no 1. Esto permite representar valores más cercanos a cero de lo que es posible cuando se asume un bit implícito de 1, pero con precisión relativa reducida respecto a los números normales.

---

4. Aritmética en precisión finita en punto flotante.

Los fenómenos que ilustramos en papel en la sección 2 se manifiestan de la misma forma en una computadora. Veamos algunos ejemplos concretos.

4.0 Preámbulo: precisión en Python/Numpy

En Python, el tipo `float` es FP64 por defecto. En Numpy se pueden usar otros formatos si se declaran explícitamente:

```
import numpy as np

x = 0.1
print(f"float nativo (FP64): {x:.20f}")

for dtype in [np.float16, np.float32, np.float64]:
    v = dtype(0.1)
    print(f"{str(dtype):>20s}: {float(v):.20f}")
```

Para visualizar el efecto de la representación, conviene imprimir con suficientes decimales usando `f"{x:.20f}"`.

4.1 Representación inexacta

```
x = 0.1
y = 0.2
z = 0.3

print(f"0.1       = {x:.20f}")
print(f"0.2       = {y:.20f}")
print(f"0.1 + 0.2 = {x+y:.20f}")
print(f"0.3       = {z:.20f}")
print(f"0.1 + 0.2 == 0.3? {x + y == z}")
```

El resultado es `False`. Ni 0.1, ni 0.2, ni 0.3 tienen representación binaria exacta. La suma se calcula sobre aproximaciones y se compara contra otra aproximación diferente.

4.2 Absorción

```
x = 1e16
y = 1
print(f"1e16     = {x:.1f}")
print(f"1e16 + 1 = {x + y:.1f}")
print(f"1e16 + 1 == 1e16? {x + y == x}")
```

Resultado: `True`. Aquí tanto 10^{16} como 1 tienen representación exacta en FP64. El problema es que a esa escala el salto entre números representables es 2, por lo que sumar 1 no cambia el valor almacenado. Es el mismo fenómeno de absorción de la sección 2.1.

4.3 Cancelación catastrófica

```
x = 1e16 + 1
y = 1e16
print(f"(1e16 + 1) - 1e16 = {x - y}")
print(f"¿Es cero? {x - y == 0}")
```

Resultado: `True`. La suma $10^{16} + 1$ ya perdió el 1 (por absorción), así que la resta devuelve 0 en lugar de 1.

4.4 Cancelación en la fórmula cuadrática

Consideremos las raíces de $x^2 - 10^9 x + 1 = 0$. Las raíces son ambas positivas, inversas entre sí (producto = 1), y una es cercana a 10^9.

```
import math

a, b, c = 1.0, -1e9, 1.0

x_grande = (-b + math.sqrt(b**2 - 4*a*c)) / (2*a)
x_chica  = (-b - math.sqrt(b**2 - 4*a*c)) / (2*a)

print(f"x_grande = {x_grande:.20f}")
print(f"x_chica  = {x_chica:.20f}")
```

Resultados:

```
x_grande = 1000000000.00000000000000000000
x_chica  = 0.00000000000000000000
```

La raíz grande se calcula correctamente, pero la raíz chica se calcula como cero: un error relativo del 100%. El problema está en que $-b - \sqrt{b^2 - 4ac}$ es una resta de dos números casi iguales (10^9 y $\approx 10^9$), produciendo cancelación catastrófica.

Una alternativa que evita la resta problemática:

```
x_chica_buena = c / (a * x_grande)
print(f"x_chica (mejorada) = {x_chica_buena:.20f}")
```

Resultado: $\approx 10^{-9}$, que es correcto. La misma operación matemática, resuelta de otra forma, da un resultado radicalmente mejor (ver sección 7).

4.5 *Overflow* y *underflow*

Al realizar operaciones que se salen del rango representable, se producen *overflow* (resultados demasiado grandes) o *underflow* (resultados demasiado pequeños).

Por ejemplo, el siguiente conjunto de operaciones produce *overflow*:

```
x= 100.
for i in range(10):
    x = x**2
for i in range(10):
    x = np.sqrt(x)
print(f"x = {x:.2e}")
```

Resultado: error. Overflow.

Mientas que el siguiente conjunto de operaciones produce *underflow*:

```
x = 0.1
for i in range(10):
    x = x**2
for i in range(10):
    x = np.sqrt(x)
print(f"x = {x:.2e}")
```

Resultado: 0.0. Aquí el underflow no es explícito, sino que se manifiesta como una pérdida total de precisión: el resultado es exactamente cero, aunque el valor real es un número positivo muy pequeño.

---

5. Condicionamiento de problemas

Al trabajar con precisión finita, debemos asumir que los datos de entrada ya tienen errores de representación. Por tanto, cuando queremos resolver un problema de cálculo, surge la pregunta: **¿qué tan sensible es el resultado del cálculo a esas perturbaciones?**

La respuesta tiene dos componentes: el **condicionamiento** del problema, que es una propiedad intrínseca del problema matemático que estamos tratando de resolver, y la **estabilidad** del algoritmo, que es una propiedad del método específico que usamos para resolverlo.

En esta sección abordaremos el concepto de condicionamiento, y en la siguiente sección hablaremos de estabilidad.

5.1 Algunos ejemplos numéricos

Consideremos una suma. Si los valores representados son $\hat{x} = x + \delta x$ y $\hat{y} = y + \delta y$ con $|\delta x| \lesssim \varepsilon_{\text{mach}}|x|$ y $|\delta y| \lesssim \varepsilon_{\text{mach}}|y|$, al sumar obtenemos:

$$\hat{z} = \hat{x} + \hat{y} = (x + y) + (\delta x + \delta y) = z + \delta z$$

Si x e y tienen el mismo signo, $|\delta z| \lesssim \varepsilon_{\text{mach}}|z|$, y el error relativo en la salida es del mismo orden que en la entrada. La suma de valores del mismo signo es lo que llamamos un problema bien condicionado: los errores de entrada no se amplifican desproporcionadamente en la salida.

Si x e y tienen signos opuestos y son cercanos en magnitud, $z = x + y$ es pequeño, pero δz no necesariamente lo es. El error relativo $|\delta z|/|z|$ puede ser mucho mayor que $\varepsilon_{\text{mach}}$. Esto es exactamente el problema de cancelación: la resta de números cercanos es un problema mal condicionado.

5.2 Caso escalar

Formalicemos la idea anterior para el caso de una función escalar $f : \mathbb{R} \to \mathbb{R}$. Es decir, dado un dato $x \in \mathbb{R}$, queremos calcular alguna salida $f(x) \in \mathbb{R}$.

A causa de la representación con precisión finita, el dato de entrada no es exactamente x, sino una perturbación $\hat{x} = x + \delta x$. Por tanto, en el mejor de los casos podemos intentar calcular $f(\hat{x})$ y no $f(x)$.

El resultado que obtenemos al calcular $f(\hat{x})$, asumiendo que δx es pequeño, y que somos capaces de calcular f con precisión arbitraria, es:

$$f(\hat{x}) = f(x + \delta x) \approx f(x) + f'(x)\,\delta x$$

Donde se usó la aproximación de primer orden de la expansión de Taylor. El error absoluto en la salida entonces es $\delta f = f(\hat{x}) - f(x) \approx f'(x)\,\delta x$.

Al factor que relaciona al error absoluto en la salida con el error absoluto en la entrada se lo llama **número de condición absoluto** del problema. En este caso es:

$$\hat{\kappa}_f(x) = |f'(x)|$$

Podemos también calcular la relación entre el error relativo en la salida y el error relativo en la entrada, definiendo el **número de condición relativo** (que en general es el que nos interesa):

$$\kappa_f(x) = \left|\frac{x \cdot f'(x)}{f(x)}\right|$$

Este número indica cuánta precisión podemos esperar perder (o ganar) al calcular $f(x)$. Como ya se dijo, este número es una propiedad intrínseca del problema, independientemente del algoritmo que usemos. Es decir, existen problemas más o menos sensibles a perturbaciones en los datos de entrada. A los problemas con κ pequeño se los llama **bien condicionados**, y a los problemas con κ grande, o excesivamente sensibles, se los llama **mal condicionados**.

Ejemplo: $f(x) = x - 1$

$$\kappa_f(x) = \left|\frac{x}{x-1}\right|$$

Cuando x está cerca de 1, el denominador es pequeño y κ crece mucho. Esto formaliza el fenómeno de cancelación: restar valores cercanos es un problema inherentemente mal condicionado.

5.3 Caso matricial

La resolución de un sistema lineal $Ax = b$ es un problema matricial natural. Si perturbamos b en δb, el cambio en la solución es:

$$\delta x = A^{-1}\delta b \implies \|\delta x\| \leq \|A^{-1}\|\|\delta b\|$$

Dividiendo por $\|x\|$ y usando $\|b\| \leq \|A\|\|x\|$:

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\|\frac{\|\delta b\|}{\|b\|}$$

El factor de amplificación es el número de condición del problema, que depende únicamente de la matriz A. Por esto se define el **número de condición de una matriz** como:

$$\kappa(A) = \|A\|\|A^{-1}\|$$

En la norma 2, esto se puede escribir en términos de los valores propios de $A^\top A$:

$$\kappa_2(A) = \sqrt{\frac{\lambda_{\max}(A^\top A)}{\lambda_{\min}(A^\top A)}}$$

Si A es normal ($A^\top A = AA^\top$), se simplifica a:

$$\kappa_2(A) = \frac{|\lambda_{\max}(A)|}{|\lambda_{\min}(A)|}$$

Es decir, el número de condición de una matriz está dado por el cociente entre su valor propio de mayor magnitud y su valor propio de menor magnitud. Esto implica que cuando la matriz tiene valores propios muy dispares, el número de condición es grande, y el sistema es sensible a perturbaciones (está mal condicionado). Otra interpretación posible es que un κ grande indica que la matriz está "cerca" de ser singular (no invertible) y que por tanto estamos "cerca" de tener un sistema de ecuaciones mal definido.

Ejemplo: un sistema casi singular

Consideremos:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4.0001 \end{bmatrix}, \qquad b = \begin{bmatrix} 3 \\ 6.0001 \end{bmatrix}$$

La solución es exactamente $x = [1, 1]^\top$. Pero si el segundo componente de b fuera 6.0002 (un cambio relativo de $\sim 1.7 \times 10^{-5}$), la solución cambia a exactamente $[-1, 2]^\top$. Es decir, un cambio pequeño en los datos produce un cambio enorme en el resultado.

Esto no significa que el algoritmo esté mal, de hecho, ni siquiera fue necesario especificar cómo resolveríamos el problema. Significa que el problema es sensible: su número de condición es grande, y ningún algoritmo puede evitar esa sensibilidad ante pequeñas perturbaciones en los datos de entrada.

---

6. Estabilidad de algoritmos

El condicionamiento del problema determina la mejor precisión que podemos esperar en el resultado. Pero, además, hay mejores y peores maneras de resolver un mismo problema. Por ejemplo, un mal algoritmo puede amplificar los errores más allá de lo que el condicionamiento del problema determina.

6.1 Estabilidad de algoritmos

Decimos que un algoritmo es **estable** si no amplifica los errores más de lo que está determinado por el condicionamiento del problema. Es decir, un algoritmo estable produce resultados con un error relativo del orden de $\kappa \cdot \varepsilon_{\text{mach}}$, donde κ es el número de condición del problema y $\varepsilon_{\text{mach}}$ la precisión de la entrada. Si mejoramos la precisión de la entrada (por ejemplo, pasando de FP32 a FP64), el error del resultado mejora proporcionalmente.

Un algoritmo **inestable** produce errores mayores que lo esperado por el condicionamiento. Mejorar la precisión de la entrada no necesariamente mejora el resultado en la proporción esperada.

Ejemplos

- **Fórmula cuadrática** (sección 4.4): la forma estándar es inestable para ciertos coeficientes; la forma alternativa vía $c/(a \cdot x_{\text{grande}})$ es estable.
- **Valores propios:** calcularlos a partir del polinomio característico (encontrar raíces de $\det(A - \lambda I) = 0$) es inestable. Los algoritmos dedicados (como el que usa `numpy.linalg.eigvals`) son estables.
- **Resolución de sistemas lineales:** la eliminación Gaussiana con pivoteo es estable; y también lo es la resolución via inversión matricial si la misma se calcula apropiadamente. Sin embargo, es en general más preciso resolver el sistema mediante eliminación Gaussiana que calcular la inversa y luego multiplicar, aunque ambos métodos sean estables.

---

7. Precisión reducida en aprendizaje automático

Durante muchos años el formato de trabajo estándar en ML fueron FP32 y más recientemente FP64, en el entendido de que la precisión más alta permitía obtener mejores resultados. La complejidad computacional de las técnicas modernas hace que sea necesario buscar la menor precisión (es decir, la menor cantidad de bits en las representaciones) que permita obtener resultados útiles, una técnica conocida como **cuantización** (*quantization*).

Hoy aparecen cada vez más configuraciones de precisión reducida:

- `float16` (FP16)
- `bfloat16` (BF16): mismos bits de exponente que FP32 (y por tanto rango similar), pero con menos bits de mantisa (7 en lugar de 23). Permite trabajar con el rango de FP32 pero con una precisión relativa algo menor que FP16, que es suficiente para muchas aplicaciones de ML.
- `FP8` y `NVFP4`: Nvidia desarrolló formatos de 8 y 4 bits para entrenamiento de redes neuronales en sus GPUs de arquitectura Hopper (ej. H100). Estos formatos sacrifican precisión a cambio de una reducción en el uso de memoria y ancho de banda, y un aumento en la cantidad de operaciones por segundo (medido típicamente en TOPS o TFLOPS: Tera (Floating Point) Operations Per Second). Por tanto, estos modelos permiten almacenar y entrenar modelos más grandes (es decir con más parámetros) en el mismo hardware. De forma general, en GPUs H100 de Nvidia los TOPS para operaciones ejecutadas sobre Tensor Cores (un conjunto de sub-procesadores específicamente diseñados para operaciones de ML en la GPU) crecen de forma inversamente proporcional a la cantidad de bits: FP32 → FP16/BF16 → FP8 → NVFP4, con un aumento de 2x en cada paso.

La siguiente imagen muestra los formatos FP16, BF16, y dos variantes de FP8 (con diferente cantidad de bits para exponente y mantisa):[3]
Ejemplo de formatos de precisión reducida. Crédito: NVIDIA

En la práctica moderna de ML se usa con frecuencia **precisión mixta**: ciertas operaciones como las multiplicaciones se hacen en baja precisión, otras como las acumulaciones en precisión mayor, y las variables sensibles se almacenan en un formato más estable.

---

1. By Fresheneesz at the English Wikipedia project, CC BY-SA 3.0 ↩
2. Tomado de Accuracy and Stability of Numerical Algorithms, Higham 2002, fig. 2.1 ↩
3. Fuente: https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html ↩